🌟 INDEX

📚 Table of Contents —>

Type	Number	Title / Name	Topics Covered
Theory	T1	**Programming Fundamentals**	Definition , Coding vs Programming , Thinking , 10-Step Mindset , Workflo
Theory	T2	**Computer Fundamentals**	IPO Cycle , Flowchart Symbols , Hardware/Software , CPU Architectu Registers & Program Execution Flow
Theory	T3	**Number Systems & Conversions**	Decimal , Binary , Octal , Hexadecima Memory Units , Step-by-Step Convers
Module	Module 4	**Basics of C++**	History , C vs C++ , Program Structur Rules to Master
Module	Module 5	**Variables, Tokens, Identifiers, Constants, Literals & Keywords**	5 Tokens, Naming Rules, Constants/ Phase Keyword Roadmap
Module	Module 6	**Data Types in C++**	Primitive , Derived , User-Defined , siz void Type
Module	Module 7	**Operators in C++**	Arithmetic , Relational , = vs == , Logi Compound , Inc/Dec , Bitwise , Terna Formula Exercise
Module	Module 8	**Process Flow & Selection Statements (if-else & Menu Programs)**	Sequential , Selection , Iterative Flow Bigger of 2 Numbers , Multiple of 7, T Checker , Menu-Driven Calculator , G Numbers
Module	Module 9	**Iterative Process Flow — while Loops**	Loop Essentials , 1 to 10 , Odd Numb Updates, Factorial, Sums, Countdown Loops, Multiplication Table
Module	Module 10	**for Loops & Number Logic**	Syntax, Execution Flow, Factors of N, Number Program
Module	Module 11	**Nested Loops & Pattern Printing**	Outer vs Inner, Square Grid, Triangles Inverted, Pyramid, Hollow U-Shape, B Coefficient nCr
Module	Module 12	**Functions in C++ & Greatest Common Divisor (GCD)**	Prototypes, Definitions, Calls, Factori Reusability, Euclidean GCD Algorithm
Module	Module 13	**switch Statement & setprecision Formatting**	switch-case Rules, Menu Calculator, Decimal Precision
Module	Module 14	**User-Defined Types — struct, union, and enum**	Struct vs Union Memory Layout, Dot Enum & Scoped enum class

Detailed Overview of Topics & Modules

- **T1: Programming Fundamentals**
  - Definition & Core Principles of Programming
  - Coding vs. Programming Distinction
  - Binary Thinking & 10-Step Problem-Solving Mindset
  - Software Development Workflow & Key Terminology
- **T2: Computer Fundamentals**
  - Input-Process-Output (IPO) Cycle
  - Flowchart Symbols & Process Mapping
  - Hardware vs. Software Integration
  - CPU Architecture: ALU, Control Unit, Registers, and Program Execution Flow
- **T3: Number Systems & Conversions**
  - Base Systems: Decimal, Binary, Octal, and Hexadecimal
  - Bits, Bytes, and Memory Storage Units
  - Step-by-Step Conversion Techniques between Bases
- **Module 4: Basics of C++**
  - History & Evolution of C++
  - Key Differences Between C and C++
  - Anatomy of a Basic C++ Program
  - 10 Golden Rules to Master C++ Syntax
- **Module 5: Variables, Tokens, Identifiers, Constants, Literals & Keywords**
  - The 5 Primary Tokens in C++
  - Identifier Naming Rules & Best Practices
  - Constants and Literal Types
  - 4-Phase Keyword Mastery Roadmap
- **Module 6: Data Types in C++**
  - Primitive, Derived, and User-Defined Data Types
  - Memory Allocation and the sizeof Operator
  - Understanding and Applying the void Type
- **Module 7: Operators in C++**
  - Arithmetic, Relational, and Logical Operators
  - Assignment vs. Equality (= vs. ==)
  - Compound Assignment & Increment/Decrement Operators
  - Bitwise & Ternary Operators
  - Practical Exercise: Heron's Formula
- **Module 8: Process Flow & Selection Statements**
  - Sequential, Selection, and Iterative Flow Controls
  - if-else Syntax & Branching Logic
  - Programs: Bigger of 2 Numbers, Multiple of 7, Triangle Type Checker, Greatest of 3 Numbers
  - Menu-Driven Programs Architecture
- **Module 9: Iterative Process Flow — while Loops**

    - Looping Essentials & Condition Maintenance
    - Basic Exercises: Printing Ranges, Odd Numbers, and Countdowns
    - Multiplication/Division Updates & Accumulator Pattern (Factorial, Sums)
    - Understanding & Preventing Infinite Loops
    - Program: Multiplication Tables
- **Module 10: for Loops & Number Logic**
    - for Loop Syntax & Execution Step Sequence
    - Factor Discovery & Number Analysis Algorithms
    - Program: Perfect Number Verification
- **Module 11: Nested Loops & Pattern Printing**
    - Outer vs. Inner Loop Mechanics
    - Grid Patterns: Squares, Right-Angled Triangles, and Inverted Triangles
    - Advanced Patterns: Pyramids, Hollow U-Shapes, and Number Grids
    - Mathematical Computation: Binomial Coefficient (nCr)
- **Module 12: Functions in C++ & GCD**
    - Function Prototypes, Definitions, and Calling Mechanisms
    - Code Reusability with Factorial and nCr Functions
    - Greatest Common Divisor (GCD) using the Euclidean Algorithm
- **Module 13: switch Statement & setprecision Formatting**
    - switch-case Syntax, Rules, and Fallthrough Behavior
    - Building Interactive Menu-Driven Calculators
    - Floating-Point Formatting with <iomanip> and setprecision
- **Module 14: User-Defined Types — struct, union, and enum**
    - Structure vs. Union Memory Layout Analysis
    - Member Access via the Dot (.) Operator Enumerations (enum) and Scoped enum class

🔴 MEMORY MANAGMENT (UNIVERSAL)⚠️

💻 *C++ Architecture, Memory & Algorithms: Inter-Function Communication, Storage Classes & Recursion*

Modular Program Design, Data Flow Topologies, Variable Lifecycles, Recursive Call Stacks (LIFO), and Complexity Analysis in C++

💡 At a Glance: Core Mental Models & Analogies

- **Modular Isolation (Functions):** Dividing a 100-line monolithic program into ten 10-line functions isolates bugs and amendments to a single 10-line block while enabling **code reusability** (like printing a receipt divider line before and after bill totals).
- **void vs. NULL (The Silence Analogy):** Being silent in **English** (a language you know, but deliberately choose not to speak right now) is **void**—data types exist, but the function chooses not to return a value. Being silent in **Japanese** (a language you do not possess at all) is **NULL**—the absence of a value/type altogether.
- **auto vs. global (Personal Wallet vs. Trip Pool):** On a group trip, each student's personal pocket money is **auto (local)**—visible only to them. The shared **Trip Budget** used to pay for fuel, group meals, DJ/events, or tire repairs is **global**—accessible and modifiable across every event function.
- **static (The Multiplayer Cave Coins):** In a two-player game, a cave starts with **100 coins**. If Player 1 enters and takes 10 coins, Player 2 entering later must see **90 coins**, not a reset to 100. Yet the coins remain hidden inside the cave until a player enters it. **static** preserves state across function calls while keeping visibility strictly **local** to that function.
- **register (Zero-Wait CPU Access):** Storing heavily accessed variables directly inside **CPU registers** eliminates memory fetch wait times, allowing the CPU to inspect and update values instantaneously.

- **Recursion & The Base Case Anchor:** A function calling itself must have a **Base Case / Anchoring Condition / Terminating Condition** defined at a **known result** (e.g., 0! = 1, 1! = 1). Without it, execution plunges into negative numbers infinitely. Recursive calls unwind in **Last-In, First-Out (LIFO) Stack** order.

---

1. 🏗️ Core Concepts: Modularity & Functions

In software engineering, a **function** is a self-contained, repeatable block of code that isolates a specific piece of logic from a larger program.

🛡️ Why Isolate Logic into Functions?

I. **Fault Isolation & Maintainability:** In a monolithic 100-to-200-line program, a single bug or requirement change forces you to inspect the entire codebase. By partitioning the program into 10-line modular functions, any error or amendment is strictly **confined to that 10-line scope**.

II. **Code Reusability:** Write the logic once and invoke it multiple times across different parts of the application (for example, rendering a formatted star-divider line on a supermarket or hotel receipt).

III. **Decoupled Architecture (UI vs. Logic):** As demonstrated in a **BMI Calculator** or modern web/game application, **input collection**, **validation/business logic**, and **output rendering** operate as independent functions that integrate seamlessly into a single user workflow.

🔍 The void Data Type vs. NULL

- **void (Having, but Not Emitting):** The language possesses data types, and the function executes valid operations, but it **deliberately chooses not to return an output value**.
- **int main() vs. void:** Notice that return 0; returns an **integer (0)**, which is why main is typed as int main(), whereas a void function returns no value at all.

Concept	Technical Meaning	Lecture Analogy	Code Behavior
void	Deliberate absence of a return value or parameter type	**Being silent in English:** You know the language, but choose not to output anything right now.	Valid function return type (void display()); cannot return a value (return; only).
NULL	Absence of an object, address, or value altogether	**Being silent in Japanese:** You do not possess the language/data in the first place.	Represents an uninitialized/empty pointer or non-existent data reference.

2. 🔄 Inter-Function Communication Paradigms

Inter-Function Communication defines how independent functions exchange data via **arguments (inputs)** passed by the caller and **return values (outputs)** sent back by the callee.

> 🔑 **Parameter Naming Rule (Formal vs. Actual Parameters):** When passing variables like x or y from main() into a function cave(int q), the parameter q inside the function header is a **brand-new local creation**. Even if you name the parameter x inside the function while passing y from main(), there is zero naming conflict because each function maintains its own isolated variable scope.

📊 The 4 Communication Topologies

Communication Type	Function Signature Pattern	Data Flow Direction	Lecture Analogy	Primary Use Case
1. No Argument, No Return	void display()	Caller —(trigger)—> Callee	Printing a decorative star line (* * * * *) on a hotel receipt.	Static UI formatting, banners, dividers, or fixed status logs.
2. No Argument, With Return	int getInput()	Caller <—(value)— Callee	A dedicated input prompt function that reads cin and hands the value back.	Input acquisition, sensor/state polling.
3. With Argument, No Return	void res(int x)	Caller —(input)—> Callee	Asking someone *"When will you return my money?"* and getting **blue ticks** (no reply).	Validation & direct display, logging, or mutating shared/static state (cave(x)).
4. With Argument, With Return	int fact(int x)	Caller <—(in/out)— > Callee	Ordering **coffee**: you pass the order	Mathematical computations, recursive

Communication Type	Function Signature Pattern	Data Flow Direction	Lecture Analogy	Primary Use Case
			(**argument**) and receive coffee back (**return value**).	functions, and data transformations.

3. 🗄️ C++ Storage Classes & Variable Lifecycles

A **Storage Class** defines four critical properties of a variable: its **storage location** (RAM vs. CPU Register), **scope/visibility** (where it can be accessed), **lifetime** (how long it stays alive in memory), and **reset behavior**.

- 1️⃣ **auto (Automatic / Local Storage Class):** Every local variable declared inside a function or block (e.g., int x = 40; or for (int i = 0; ...)) is an auto variable by default. It is automatically **created** when entering the block and automatically **destroyed** when exiting the block.
- 2️⃣ **global (Global Storage Class):** Declared **outside of all functions**. Accessible and modifiable by every function in the program (like a shared **Group Trip Budget** deducted by fuel(), eat(), and event()).
- 3️⃣ **static (Persistent Local Storage Class):** Declared inside a function using the **static** keyword (e.g., static int coins = 10;). Initialized **only once**; it **does not die** when the function exits, retaining its updated value across calls while remaining strictly **invisible outside the function** (like coins hidden inside a game cave).
- 4️⃣ **register (CPU Register Storage Class):** Requests storage directly inside a **CPU Register** so the CPU has **zero wait time** and can inspect/update the value immediately.
- 5️⃣ **extern (External Storage Class):** Declares a global variable defined across multiple files (covered in advanced sessions).

📊 Structured Reference Table: C++ Storage Classes

Storage Class	Keyword / Placement	Memory Location	Scope (Visibility)	Lifetime	Reset Behavior on Re-entry	Lecture Analogy
Automatic	auto (default inside {})	RAM (Stack)	Local to defining block/function	Destroyed upon exiting block {}	Re-created & re-initialized every call	**Personal Wallet / Roll Number:** Specific to one person/block only.
Global	Outside all functions	RAM (Data Segment)	Entire program (all functions)	Entire program execution	Retains mutations across all functions	**Common Trip Budget:** Shared pool for fuel, food, DJ, and repairs.
Static	static (inside function)	RAM (Data Segment)	Local to defining function only	Entire program execution	**Initialized once;** retains updated value	**Cave Coins in a Game:** Hidden inside the

Storage Class	Keyword / Placement	Memory Location	Scope (Visibility)	Lifetime	Reset Behavior on Re-entry	Lecture Analogy
					across calls	cave, remembers coins taken.
Register	register	**CPU Register**	Local to defining block	Destroyed upon exiting block {}	Re-created every call (zero CPU wait time)	**Direct CPU Scratchpad:** Instant CPU update without memory latency.
External	extern	RAM (Data Segment)	Across multiple source files	Entire program execution	Shared across linked files	**Multi-Module Shared State**.

4. 🖥️ VS Code-Style Implementations (Functions & Storage Classes)

📄 Program 1: Modular Receipt Divider & Pass/Fail Evaluator

```cpp
//
================================================================================
===

// File: inter_function_comm.cpp

// Topic: Modular Functions, Reusability, and Void Communication Patterns

//
================================================================================
===



#include <iostream>

using namespace std;



// --- 1. Function Definitions ---



// Pattern 1: No Argument, No Return (Reusable UI Divider)

void display() {

    cout << endl;

    for (int i = 0; i < 10; i++) {

        cout << "* ";          // Space after '*' increases visual distinction
```

```
    }

    cout << endl;

}



// Pattern 3: With Argument, No Return (Evaluates & Prints Result Directly)

void res( int x ) {

    if ( x >= 70 ) {

        cout << "Result: Pass";

    } else {

        cout << "Result: Passed Away (Fail)";

    }

}



// --- 2. Main Execution —



int main() {

    int x;
```

```
    cout << "Enter student marks: ";

    cin  >> x;

    display();                // Print top receipt border

    res(x);                   // Evaluate and print pass/fail status

    display();                // Reuse border function for bottom line

    return 0;

}
```

📄 Program 2: Group Trip Expense Tracker (global vs. auto)

```
//
================================================================================
===

// File: trip_budget_global.cpp

// Topic: Global Storage Class vs. Local (Auto) Variables
```

```
//
================================================================================
===



#include <iostream>

using namespace std;



// --- 1. Global State Declaration ---

int money;                  // Global variable: visible across ALL functions



// --- 2. Helper & Event Functions ---



void b() {

    cout << "Remaining Balance: " << money << endl;

}



void bal() {

    cout << "\n";
```

```
    for (int i = 0; i < 20; i++) {

        cout << "*";

    }

    cout << "\n";

    b();                    // Inter-function call to print balance

}



void fuel() {

    int x;                  // Local (auto) variable: destroyed when fuel() ends

    cout << "Enter Fuel Cost:  ";

    cin  >> x;

    money -= x;             // Mutates global trip budget

}



void eat() {

    int x;                  // Distinct local 'x', no conflict with fuel()'s 'x'

    cout << "Enter Food Cost:  ";
```

```
    cin  >> x;

    money -= x;

}



void event() {

    int x;

    cout << "Enter Event Cost: ";

    cin  >> x;

    money -= x;

}



// --- 3. Main Execution ---



int main() {

    cout << "Enter Initial Trip Budget: ";

    cin  >> money;
```

```
    fuel();

    bal();                  // Check balance after fuel



    eat();

    bal();                  // Check balance after eating



    event();

    bal();                  // Check final balance after event



    return 0;

}
```

Program 3 : Multiplayer Game Cave (static Storage Class + Homework Extension)

```
//
=============================================================================
===
```

```cpp
// File: game_cave_static.cpp

// Topic: Static Storage Class & Homework Solution (Bag Capacity + Stock Check)

//
==========================================================================
===




#include <iostream>

using namespace std;



// --- 1. Cave Logic (Static State Encapsulation) ---



void cave(int q, int bagCapacity) {

    // 'static' initializes 'coins' ONLY ONCE on the first function call.

    // Its value persists across subsequent calls but is invisible outside cave().

    static int coins = 10;
```

```
    cout << "\n----------------------------------------\n";

    cout << "Entry Balance in Cave   : " << coins << " coins\n";

    cout << "Player Requested        : " << q     << " coins (Bag Limit: " << bagCapacity << ")
\n";




    // Homework Condition 1: Cap request at the player's available bag capacity

    int allowedRequest = (q > bagCapacity) ? bagCapacity : q;

    if (q > bagCapacity) {

        cout << "[Notice] Bag full! Request capped at bag capacity (" << bagCapacity << ").
\n";

    }




    // Homework Condition 2: Prevent negative cave balance if request exceeds stock

    if (allowedRequest > coins) {

        cout << "[Warning] Only " << coins << " coins available! Allocating remaining " <<
coins << " coins.\n";

        coins = 0;
```

```
    } else {

        coins -= allowedRequest;

    }



    cout << "Updated Balance in Cave : " << coins << " coins\n";

    cout << "-----------------------------------\n";

}



// --- 2. Main Execution ---



int main() {

    int x, y;

    cout << "Enter coins requested by Player 1 and Player 2: ";

    cin  >> x >> y;
```

```
    cave(x, 10);              // Player 1 enters cave (Bag capacity = 10)

    cave(y, 4);               // Player 2 enters SAME cave later (Bag capacity = 4)



    return 0;

}
```

5. 🔁 Recursion: Core Theory, Recurrence Relations & Stack Mechanics

📐 Formal Definitions

- **Recursion:** Calling a function within the **same function** is known as **recursion**.
- **Recursive Function:** Any function that invokes itself directly or indirectly is called a **recursive function**.
- **Base Case / Anchoring Condition / Terminating Condition:** Every recursive function must include a stopping condition to terminate execution. These three terms—**Base Case**, **Anchoring Condition**, and **Terminating Condition**—are completely synonymous.

⚓ The "Known Result" Rule for Writing Base Cases

Where should you place the Base Case / Anchoring Condition? Always anchor

your recursion at a **Known Result**—an input value for which you already know the answer directly without any computation.

- ⇨ **Factorial Example:** For which values do we already know the factorial without computing? We know directly that **1! = 1** and **0! = 1**. Therefore, if (x == 0 || x == 1) return 1; forms our exact anchoring condition.
- ⇨ **What happens if the Base Case is omitted?** Without a stopping mechanism, 1 * fact(0) calls 0 * fact(-1), which calls -1 * fact(-2), -2 * fact(-3), and plunges into negative numbers in an **infinite execution loop** (causing a stack overflow crash).

🧮 Mathematical Model & Recurrence Relation (Factorial)

- **Mathematical Model:** n! = n * (n - 1)!
- **Recurrence Relation (Function Terminology):** fact(n) = n * fact(n - 1)
- **Progressive Expansion:**
  - fact(n) = n * fact(n - 1)
  - fact(n - 1) = (n - 1) * fact(n - 2)
  - fact(n - 2) = (n - 2) * fact(n - 3) ... continuing until n reaches the known result (1 or 0).

---

6. 🔍 Step-by-Step Execution Trace & LIFO Call Stack Analysis (fact(5))

When main() calls fact(5), execution proceeds in two distinct phases: **Phase 1 (Top-Down Winding / Calling)** and **Phase 2 (Bottom-Up Unwinding / Returning)**.

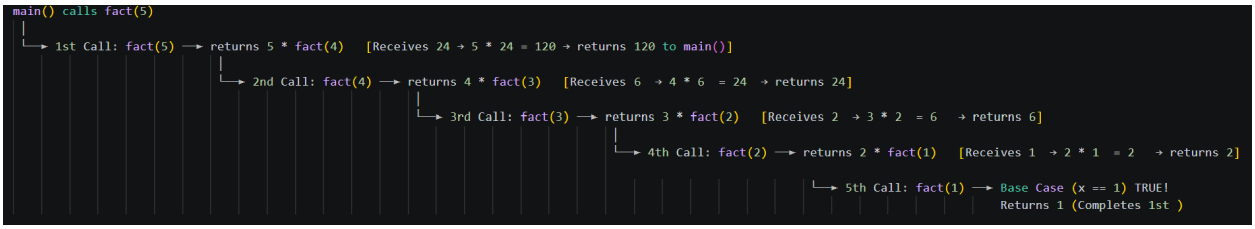
📚 **Stack Behavior (LIFO — Last-In, First-Out):** Out of the 5 recursive invocations (fact(5) down to fact(1)), the **5th call (fact(1)) is the last function called, and it is the first function to complete its execution**. Whenever a function finishes execution, **program control returns to its caller**, replacing the function call expression with its returned value.

📊 Complete Execution & Return Flow Table for n = 5

Call #	Active Call & Parameter	Base Case Check (x == 0 || x == 1)	Action During Winding (Call Flow)	Return Unwinding (LIFO Completion Order)
1st Call	fact(5) (x = 5)	5 == 0 || 5 == 1 → **False**	Pauses at return 5 * fact(4); calls fact(4)	**5th to finish:** Computes 5 * 24 = 120; returns **120** to main().
2nd Call	fact(4) (x = 4)	4 == 0 || 4 == 1 → **False**	Pauses at return 4 * fact(3); calls fact(3)	**4th to finish:** Replaces fact(3) with 6; computes 4 * 6 = 24; returns **24** to 1st Call.
3rd Call	fact(3) (x = 3)	3 == 0 || 3 == 1 → **False**	Pauses at return 3 * fact(2); calls fact(2)	**3rd to finish:** Replaces fact(2) with 2; computes 3 * 2 = 6; returns **6** to 2nd Call.
4th Call	fact(2) (x = 2)	2 == 0 || 2 == 1 → **False**	Pauses at return 2 *	**2nd to finish:** Replaces

Call #	Active Call & Parameter	Base Case Check (x == 0 || x == 1)	Action During Winding (Call Flow)	Return Unwinding (LIFO Completion Order)
			fact(1); calls fact(1)	fact(1) with 1; computes 2 * 1 = 2; returns **2** to 3rd Call.
5th Call	fact(1) (x = 1)	1 == 0 || 1 == 1 → **True!** (Anchor Hit)	Executes return 1; (No further recursive call)	**1st to finish (LIFO):** Immediately returns **1** to its caller (4th Call).

🗺️ Visual Call & Return Tree Diagram



7. ⏱️ Time Complexity Comparison: Iterative vs. Recursive

Metric / Dimension	Iterative Factorial (Loop Approach)	Recursive Factorial (Self-Calling Approach)
Control Mechanism	Single for loop running from i = 1 to x (n iterations).	n nested function calls (from fact(n) down to fact(1)).
Work Done per Step	1 multiplication + loop increment per iteration = **Constant work O(1)**.	1 if condition check (1 unit) + 1 multiplication (1 unit) = **Constant work O(1)** per call.
Total Operations for Input n	n iterations × constant work = n * c	n function calls × constant work = n * c
Time Complexity	**O(n) (Order of n)**	**O(n) (Order of n)** — Identical time complexity to iteration!
Termination Mechanism	Loop condition i <= x becomes false.	Base Case / Anchoring Condition (x == 0 || x == 1) becomes true.

8. 🖥️ VS Code-Style Implementations (Iteration vs. Recursion)

📄 Program 4 : Iterative Factorial (Normal Function with loop)

```cpp
//
================================================================================
// File: factorial_iterative.cpp
// Topic: Normal Function Using Iteration (For Loop) — Time Complexity: O(n)
//
================================================================================

#include <iostream>
using namespace std;

// --- 1. Iterative Function Definition ---

int fact(int x) {
    int f = 1;
    // 1 loop from 1 to x -> O(n) Time Complexity
    for (int i = 1; i <= x; i++) {
        f = f * i;
    }
    return f;
}

// --- 2. Main Execution ---

int main() {
    int n;
    cin >> n;

    int res = fact(n);          // Calls iterative factorial function
    cout << res;

    return 0;
}
```

📄 Program 5 : Recursive Factorial (Base Case + Recurrence Relation)

```
//
================================================================================
===

// File: factorial_recursive.cpp

// Topic: Recursive Function with Base Case — Time Complexity: O(n)

//
================================================================================
===




#include <iostream>

using namespace std;




// --- 1. Recursive Function Definition ---




int fact(int x) {

    // Base Case / Anchoring Condition / Terminating Condition (Known Results)
```

```
    if (x == 0 || x == 1) {

        return 1;              // 1 unit of time ; halts recursive descent

    }



    // Recursive Call (Recurrence Relation: fact(x) = x * fact(x - 1))

    return x * fact(x - 1);     // 1 unit of time (product) + next function call

}



// --- 2. Main Execution ---



int main() {

    int n;

    cin >> n;



    int res = fact(n);          // For n = 5, receives 120 after LIFO unwinding
```

```
    cout << res;              // Prints 120




    return 0;

}
```

★ **Foreground (UI/UX) vs. Background (Core Engine):** Applications use **JavaScript / React** for frontend visuals, while high-performance engines (like **GTA V**) rely on **C++** for math, physics, and memory control.

★ **C++ vs. DSA Overlap & Upcoming Syllabus:** Storage classes belong specifically to C++ programming, whereas **Recursion** is a foundational concept shared between C++ and **Data Structures & Algorithms (DSA)**. Upcoming modules include **OOPs**, **DSA**, and **STL (Standard Template Library)** for competitive programming.

★ **Classroom & CLA-2 Logistics:** Classes from next week run until **5:00 PM**. **CLA-2** will occur next week or the following week, covering all topics taught in class up to the exam date.

📝 Quick Exam Checklist

➢ [ ] **Define Recursion & Base Case Synonyms:** State clearly that a base case is also called an **anchoring condition** or **terminating condition**, and explain why it is placed at a **known result** (0! = 1, 1! = 1).

- [ ] **Explain Missing Base Case Behavior:** Show how omitting the base case causes fact(1) → fact(0) → fact(-1) → fact(-2), resulting in infinite execution.
- [ ] **Trace LIFO Stack Unwinding:** For fact(5), demonstrate why the 5th call (fact(1)) finishes first (Last-In, First-Out Stack behavior) and how each completed call returns control and its product to its caller.
- [ ] **Prove O(n) Time Complexity for Recursion:** State that there are n function calls and each call performs a constant amount of work (1 comparison + 1 multiplication), yielding n * O(1) = O(n)—the exact same time complexity as the iterative loop.
- [ ] **Master Storage Classes & Inter-Function Topologies:** Review auto, global, static, register, extern, and the 4 argument/return communication patterns.

😍 PART - 1

💻 C++ *NOTES* (COMPLETE GUIDE)

Level: Beginner to Core C++ Foundations (Clean, ATS-Friendly & Detailed Edition)

📱 **Have Questions or Need Help?**
Get direct 1-on-1 assistance with these C++ notes: **[💬 Connect via WhatsApp for 1:1 Queries]**



@SYNEXIS69

📞 Contact & Support

- **Email :** sahil81255@gmail.com
- **Phone :** +91 8125584420

📞 Contact & Support

- **Email :** synexis96@gmail.com
- **Phone :** +91 7680876505

💡 **At a Glance: Executive Blueprint**
This textbook guide bridges computer theory and core C++ syntax. Key Learning Path: `Programming`

> `Fundamentals → Computer Architecture & Memory → Number Systems → Data Types & Tokens → Operators → Selection & Loops → Functions → User-Defined Types (Struct/Union/Enum)`.
> Designed for clarity, speed, and technical depth.

1. 🏗️ Theory T1: Programming Fundamentals

1. What is Programming?

- **Simple Definition:** Programming is teaching a computer how to perform a task step by step.
- Because a computer cannot think on its own, a programmer must give it clear, logical, and exact instructions to solve a problem.

> 💡 **Intuition: Programming vs Coding**
> Coding is typing syntax in a specific language (like C++). **Programming is the engineering process** of problem formulation, algorithm architecture, code implementation, testing, and optimization.

2. Programming vs. Coding

Many beginners think programming and coding are the same, but coding is just one part of programming:

Feature	Programming	Coding
Core Meaning	Solving a complete problem from start to finish	Writing instructions in a programming language
Activities Included	Planning, designing logic, testing, and debugging	Only writing source code
Main Focus	Focuses on **logic** and problem-solving	Focuses on **syntax** (grammar rules of the language)
Scope	Bigger, complete process	Smaller part of programming

3. How Does a Computer Think?

- A computer does **not** understand English or any human language.

- It understands only **Binary (`0` and `1`)**, where `0` means electricity OFF and `1` means electricity ON.
- To run your C++ program, a translator called a **Compiler** converts your C++ code into binary machine code:

```
// --- C++ Source Code to CPU Execution Diagram ---
Your C++ Code (Source Code)
        ↓
Compiler (Translates C++ into 0s and 1s)
        ↓
Machine Code (Binary: 0s and 1s)
        ↓
CPU (Executes binary instructions)
        ↓
Output (Displayed on screen)
```

🧠 4. Programmer's 10-Step Thinking Process

Professional programmers rarely start by typing code immediately. Instead, they follow this 10-step mindset:

1. **Understand the problem** clearly.
2. **Identify the inputs** (what data is given).
3. **Determine the expected output** (what result is needed).
4. **Break the problem** into smaller, manageable steps.
5. **Write an algorithm** (step-by-step logical solution).
6. **Draw a flowchart** if needed to visualize the logic.
7. **Write the code** in C++.
8. **Test with different inputs** (including edge cases).
9. **Fix mistakes (debugging)** if any errors occur.
10. **Improve and optimize** the solution.

Why follow this? This structured approach reduces errors, saves time, and produces cleaner, bug-free programs.

5. Programming Workflow

```
┐Problem Definition → System Analysis → Algorithm Design → Flowchart
Mapping → C++ Coding → Compilation → Debugging → Execution → Validation →
Maintenance
```

🧠 6. Key Terms Every Beginner Must Know

Term	Simple Meaning
Program	A set of instructions given to a computer to perform a task
Programming	The complete process of designing, writing, testing, and maintaining programs
Programmer	A person who writes and maintains computer programs
Source Code	Human-readable code written by the programmer (e.g., `main.cpp`)
Machine Code	Binary instructions (`0`s and `1`s) executed directly by the CPU
Compiler	A program that translates source code into machine code all at once
Algorithm	A step-by-step logical procedure to solve a problem
Flowchart	A visual/diagrammatic representation of an algorithm using standard shapes
Debugging	The process of finding and fixing errors (bugs) in a program

😆 PART - 2

🖥️ *Theory T2 : Computer Fundamentals*

Golden Rule: *"C++ is close to the hardware. The better you understand the computer, the better you will understand C++."*

> ⚡ **Golden Rule: Hardware-C++ Relationship**
> "C++ is close to the hardware. The better you understand the computer architecture, memory models, and execution flow, the better you will write efficient C++ programs."

1 . What is a Computer & IPO Cycle

- **Definition:** A computer is an electronic machine that accepts **input**, **processes** it according to instructions, **stores** data, and produces **output**.

```
// --- The IPO Storage Cycle ---
Input (Keyboard/Mouse) → Processing (CPU Logic) → Output (Monitor)
                                ↓
                        Storage (RAM / SSD)
```

🧠 2 . Flowcharts (Visualizing Program Logic)

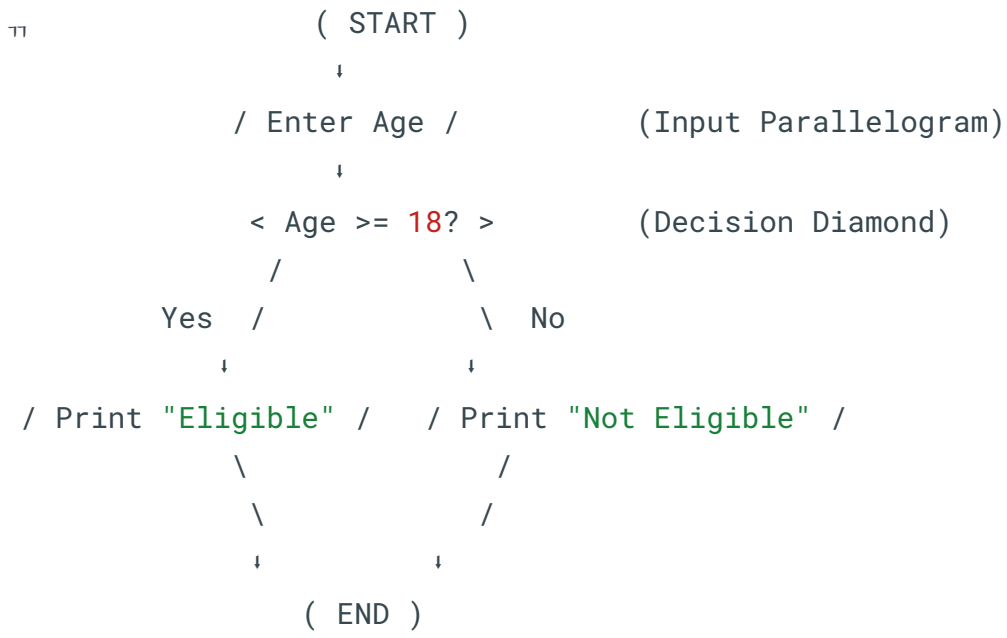
A **Flowchart** is a picture of the separate steps of a process in sequential order.

Standard Flowchart Symbols

Symbol Shape	Name	Purpose / Usage
Oval / Capsule	Start / End (Terminal)	Represents the beginning (`START`) or end (`END`) of a program
Parallelogram	Input / Output	Used for taking input (`cin`) or displaying output (`cout`)
Rectangle	Process	Used for calculations, assignments, or processing (`sum = a + b`)
Diamond	Decision	Used for conditions (`Yes/True`

Symbol Shape	Name	Purpose / Usage
		or `No/False`, like `Age > 18`)
Hexagon	Preparation	Used for setting up or initializing variables/loops
Small Circle	Connector	Joins different parts of a flowchart together
`Arrow (→, ↓)`	Flow Line	Shows the direction of process flow

Flowchart Logic: Voting Eligibility Check



🖥️ 3. Hardware vs. Software Relationship

- **Hardware:** Physical parts of the computer you can touch (CPU, RAM, SSD, Keyboard, Monitor).
- **Software:** Programs and instructions that tell the hardware what to do (VS Code, Windows/macOS, your C++ program).

```
User → Application Software (VS Code) → Operating System (OS) → Computer Hardware (CPU/RAM/SSD)
```

🧠 4. Main Components of a Computer & Real-Life Analogy

A computer mainly consists of the **CPU**, **RAM (Primary Memory)**, **Storage (SSD/HDD)**, and **Input/Output Devices**.

Every C++ statement (such as int sum = a + b;) eventually reaches the CPU.

Computer Component	Role in Computing	Human Body Analogy
Computer	Complete system	**Human Body**
CPU (Central Processing Unit)	Executes instructions and controls operations	**Brain**
RAM (Random Access Memory)	Temporary, fast working memory (cleared when power is off)	**Short-Term Memory**
SSD / Hard Drive	Permanent storage for files and installed programs	**Long-Term Memory**
Keyboard / Mouse	Takes input from the user	**Hands / Senses**
Monitor	Displays the output visually	**Eyes / Face**

5. Internal Parts of the CPU

The CPU ("Brain of the Computer") has three main internal components:

1. **ALU (Arithmetic Logic Unit):**
    - Performs all **mathematical calculations** (`+`, `-`, `*`, `/`, `%`) and **logical comparisons** (`>`, `<`, `==`).
    - *Example:* In `10 + 5 = 15`, the ALU performs the addition.
2. **CU (Control Unit):**
    - Controls and coordinates everything inside the computer.
    - Reads instructions from memory, decides what happens next, and sends control signals to hardware.
    - *Analogy:* Think of the Control Unit as a **traffic police officer** directing every operation.
3. **Registers:**
    - Very tiny, **extremely fast** memory locations located directly inside the CPU.
    - They temporarily hold numbers, memory addresses, and intermediate calculation results.
    - Registers are much faster than RAM.

6. How a C++ Program Runs (Practical Step-by-Step Flow)

```cpp
// File: main.cpp - Basic Program Execution Demo
#include <iostream>
using namespace std;

int main() {
    cout << "Hello World";
    return 0;
}
```

```
// --- Execution Lifecycle ---
1. Write Code in Editor (VS Code) → main.cpp
        ↓
2. File Saved to Storage (SSD)
        ↓
3. Compiler Translates C++ to Machine Binary (0s & 1s)
        ↓
4. Executable Binary Created (.exe / a.out)
        ↓
5. OS Loads Binary into Memory (RAM)
        ↓
6. CPU Fetches & Decodes Instructions (CU)
        ↓
7. CPU Executes Operations (ALU & Registers)
        ↓
8. Output Rendered to Display Screen
```

🥳 PART - 3

3. 🔢 Theory T3: Number Systems & Conversions

1. One Number — Four Base Systems

Humans count in Base-10 (Decimal), but computers work with electricity (`OFF = 0` , `ON = 1`), so they use Base-2 (Binary). Octal and Hexadecimal are compact shorthand representations of binary.

Number System	Base (Radix)	Digits Used
Decimal	Base 10	`0, 1, 2, 3, 4, 5, 6, 7, 8, 9`
Binary	Base 2	`0, 1`
Octal	Base 8	`0, 1, 2, 3, 4, 5, 6, 7`
Hexadecimal	Base 16	`0–9` and `A, B, C, D, E, F` (`A=10` ... `F=15`)

2. Bits, Bytes, and Memory Units

- **What is a Bit?** "Bit" stands for **Binary Digit**. It is the smallest unit of data in a computer and can hold only `0` (OFF / 0V) or `1` (ON / 5V).
- **What is a Byte?** One Byte is a group of **8 Bits** (for example: `10101100`). One `char` in C++ takes 1 Byte.

Memory Unit	Equal To
1 Bit	`0` or `1`
1 Nibble	`4 Bits`
1 Byte	`8 Bits`
1 Kilobyte (KB)	`1024 Bytes`
1 Megabyte (MB)	`1024 KB`
1 Gigabyte (GB)	`1024 MB`
1 Terabyte (TB)	`1024 GB`

3. Number System Conversions (Step-by-Step)

A. Decimal System Place Values (Base-10)

Each digit in a decimal number is multiplied by a power of `10`:

- **Whole Number Example (`3456`):**
  - `3456` = $(3 \times 10^3) + (4 \times 10^2) + (5 \times 10^1) + (6 \times 10^0)$
  - `3456` = 3000 + 400 + 50 + 6 = 3456
- **Fractional Number Example (`73.26`):**
  - Digits *after* the decimal point use *negative* powers of 10 ($10^{-1} = 0.1$, $10^{-2} = 0.01$):
  - `73.26` = $(7 \times 10^1) + (3 \times 10^0) + (2 \times 10^{-1}) + (6 \times 10^{-2})$ = 70 + 3 + 0.2 + 0.06

B. Converting Decimal to Binary (Repeated Division by 2)

Rule: Divide the decimal number by `2` repeatedly, record the remainder (`0` or `1`) at each step, and read the remainders from bottom to top (MSB to LSB).

Example 1: Convert Decimal 25 to Binary

Division Step	Quotient	Remainder
`25 ÷ 2`	`12`	**1**
`12 ÷ 2`	`6`	**0**
`6 ÷ 2`	`3`	**0**
`3 ÷ 2`	`1`	**1**
`1 ÷ 2`	`0`	**1**

- Reading remainders from **bottom to top**: **`25` in Decimal = `11001` in Binary**.

Example 2: Convert Decimal 44 to Binary

Division Step	Quotient	Remainder
`44 ÷ 2`	`22`	**0**
`22 ÷ 2`	`11`	**0**
`11 ÷ 2`	`5`	**1**
`5 ÷ 2`	`2`	**1**
`2 ÷ 2`	`1`	**0**
`1 ÷ 2`	`0`	**1**

- Reading remainders from **bottom to top**: **`44` in Decimal = `101100` in Binary**.

C. Converting Binary to Decimal (Powers of 2 Position Weights)

Rule: Multiply each binary digit by 2^position (starting from $2^0 = 1$ on the right) and add them up.

- **Powers of 2 from right to left:** $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$, $2^7 = 128$

Example 1: Convert Binary 10110 to Decimal

- Bit positions from left to right:
  - $(1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)$
  - $= (1 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1)$
  - $= 16 + 0 + 4 + 2 + 0 =$ **22 (Decimal)**

Example 2: Convert Binary 1101 to Decimal

- $(1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$
- $= 8 + 4 + 0 + 1 =$ **13 (Decimal)**

D. Complete Reference Table (0 to 15 Across All 4 Systems)

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	10
9	1001	11
10	1010	12
11	1011	13
12	1100	14
13	1101	15
14	1110	16

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)
15	1111	17

3. 🔢 Theory T3: Number Systems & Conversions

1. One Number — Four Base Systems

Humans count in Base-10 (Decimal), but computers work with electricity (`OFF = 0`, `ON = 1`), so they use Base-2 (Binary). Octal and Hexadecimal are compact shorthand representations of binary.

Number System	Base (Radix)	Digits Used
Decimal	Base 10	`0, 1, 2, 3, 4, 5, 6, 7, 8, 9`
Binary	Base 2	`0, 1`
Octal	Base 8	`0, 1, 2, 3, 4, 5, 6, 7`
Hexadecimal	Base 16	`0–9` and `A, B, C, D, E, F` (`A=10` ... `F=15`)

2. Bits, Bytes, and Memory Units

- **What is a Bit?** "Bit" stands for **Binary Digit**. It is the smallest unit of data in a computer and can hold only `0` (OFF / 0V) or `1` (ON / 5V).
- **What is a Byte?** One Byte is a group of **8 Bits** (for example: `10101100`). One `char` in C++ takes 1 Byte.

Memory Unit	Equal To
1 Bit	`0` or `1`
1 Nibble	`4 Bits`
1 Byte	`8 Bits`
1 Kilobyte (KB)	`1024 Bytes`
1 Megabyte (MB)	`1024 KB`
1 Gigabyte (GB)	`1024 MB`
1 Terabyte (TB)	`1024 GB`

3. Number System Conversions (Step-by-Step)

A. Decimal System Place Values (Base-10)

Each digit in a decimal number is multiplied by a power of `10`:

- **Whole Number Example (`3456`):**
  - `3456` = $(3 \times 10^3) + (4 \times 10^2) + (5 \times 10^1) + (6 \times 10^0)$
  - `3456` = 3000 + 400 + 50 + 6 = 3456
- **Fractional Number Example (`73.26`):**
  - Digits *after* the decimal point use *negative* powers of 10 ($10^{-1} = 0.1$, $10^{-2} = 0.01$):
  - `73.26` = $(7 \times 10^1) + (3 \times 10^0) + (2 \times 10^{-1}) + (6 \times 10^{-2})$ = 70 + 3 + 0.2 + 0.06

B. Converting Decimal to Binary (Repeated Division by 2)

Rule: Divide the decimal number by `2` repeatedly, record the remainder (`0` or `1`) at each step, and read the remainders from bottom to top (MSB to LSB).

Example 1: Convert Decimal 25 to Binary

Division Step	Quotient	Remainder
`25 ÷ 2`	`12`	**1**
`12 ÷ 2`	`6`	**0**
`6 ÷ 2`	`3`	**0**
`3 ÷ 2`	`1`	**1**
`1 ÷ 2`	`0`	**1**

- Reading remainders from **bottom to top**: **`25` in Decimal = `11001` in Binary**.

Example 2: Convert Decimal 44 to Binary

Division Step	Quotient	Remainder
`44 ÷ 2`	`22`	**0**
`22 ÷ 2`	`11`	**0**
`11 ÷ 2`	`5`	**1**
`5 ÷ 2`	`2`	**1**
`2 ÷ 2`	`1`	**0**
`1 ÷ 2`	`0`	**1**

- Reading remainders from **bottom to top**: **`44` in Decimal = `101100` in Binary**.

C. Converting Binary to Decimal (Powers of 2 Position Weights)

Rule: Multiply each binary digit by 2^position (starting from $2^0 = 1$ on the right) and add them up.

- **Powers of 2 from right to left:** $2^0 = 1$, $2^1 = 2$, $2^2 = 4$, $2^3 = 8$, $2^4 = 16$, $2^5 = 32$, $2^6 = 64$, $2^7 = 128$

Example 1: Convert Binary 10110 to Decimal

- Bit positions from left to right:
  - $(1 \times 2^4) + (0 \times 2^3) + (1 \times 2^2) + (1 \times 2^1) + (0 \times 2^0)$
  - $= (1 \times 16) + (0 \times 8) + (1 \times 4) + (1 \times 2) + (0 \times 1)$
  - $= 16 + 0 + 4 + 2 + 0 =$ **22 (Decimal)**

Example 2: Convert Binary 1101 to Decimal

- $(1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0)$
- $= 8 + 4 + 0 + 1 =$ **13 (Decimal)**

D. Complete Reference Table (0 to 15 Across All 4 Systems)

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)
0	0000	0
1	0001	1
2	0010	2
3	0011	3
4	0100	4
5	0101	5
6	0110	6
7	0111	7
8	1000	10
9	1001	11
10	1010	12
11	1011	13
12	1100	14
13	1101	15
14	1110	16

Decimal (Base 10)	Binary (Base 2)	Octal (Base 8)
15	1111	17

☺️ PART - 4

4. 🚀 Module 4: Basics of C++

1. Introduction to C++

- **Creator: Bjarne Stroustrup** developed C++ at Bell Labs (starting in 1979) as an extension to the C language (*"C with Classes"*).
- **Why C++?** It combines the fast hardware-level speed of C with **Object-Oriented Programming (OOP)** features, making it ideal for competitive programming, game engines, operating systems, and high-performance applications.

2. C vs. C++ Comparison

Feature	C Language	C++ Language
Header File for I/O	`#include <stdio.h>`	`#include <iostream>`
Output Statement	`printf("SAHIL");`	`cout << "SAHIL";`
Input Statement	`scanf("%d", &a);`	`cin >> a;`
Format Specifiers	Required (`%d`, `%f`, `%c`, `%s`)	**Not required** (`cout` and `cin` detect types automatically)
Operators Used	Function calls	`<<` (Insertion operator) and `>>` (Extraction operator)
Programming Style	Procedural Programming	Procedural + Object-Oriented Programming (OOP)

3. Basic Structure of a C++ Program

```cpp
// File: hello.cpp - Structure of a basic C++ program
#include <iostream>          // Header file for stream I/O
using namespace std;         // Standard namespace declaration

int main() {                 // Program entry point
    cout << "SAHIL";         // Print statement
    return 0;                // Return code 0 = Success
}
```

Line-by-Line Explanation:

- `#include <iostream>`: Preprocessor directive that includes the standard **Input/Output Stream** library so we can use `cin` and `cout`.

- `using namespace std;`: Tells the compiler to use the standard (`std`) namespace so we can write `cout` instead of `std::cout`.
- `int main()`: The main starting function of every C++ program. Execution **always** begins inside `main()`.
- `{ }`: Curly braces mark the start and end of the function's body.
- `cout << "SAHIL";`: Prints `"SAHIL"` on the screen. Every C++ statement ends with a semicolon (`;`).
- `( return 0; )` ▪ Tells the operating system that the program finished successfully without errors.

4. 10 Golden Rules to Master in C++

1. **Semicolon Rule:** Every executable statement in C++ must end with a semicolon (`;`).
2. **Case Sensitivity:** C++ is case-sensitive (`age`, `Age`, and `AGE` are treated as three different names; keywords like `int`, `if`, `return` must be lowercase).
3. **Starting Point:** Program execution always begins at `int main()`.
4. **Always Initialize Variables:** Local variables that are not given a starting value hold random **garbage values** (e.g., always set `int sum = 0;` before adding in a loop).
5. **Integer Division Rule:** Dividing two integers (`int / int`) always drops the decimal part (e.g., `10 / 3` gives `3`, not `3.33`).
6. **`=` vs `==` Rule:** Single `=` assigns/stores a value (`a = 5`), whereas double `==` compares two values (`a == 5`).
7. **Modulus (`%`) Rule:** The remainder operator `%` works **only with integers**, never with `float` or `double`.
8. **Braces `{ }` Scope Rule:** Always pair every opening brace `{` with a closing brace `}` to define code blocks cleanly.
9. **Zero-Based Indexing:** Counting in arrays and strings starts from `0`, not `1`.
10. **Use Comments & Good Names:** Use `//` for single-line comments and `/* ... */` for multi-line comments to explain your code.

😌 PART - 5

5. 🔑 Module 5: Variables, Tokens, Identifiers, Constants, Literals & Keywords

1. What is a Variable?

- A **Variable** is a named container (memory location in RAM) used to store data whose value can change while the program runs.
- **Syntax:** `data_type variable_name = value;`
- **Example:**int age = 20;
    - `int` → Data type (stores whole numbers)
    - `age` → Variable name (Identifier)
    - `=` → Assignment operator
    - `20` → Literal value stored in memory

2. What are Tokens in C++?

A **Token** is the smallest individual unit in a C++ program that is meaningful to the compiler. C++ has **5 types of tokens**:

1. **Keywords:** Reserved words with fixed meanings (`int`, `if`, `while`, `return`).
2. **Identifiers:** Names given by the programmer to variables, functions, structures, etc. (`age`, `sum`, `main`).
3. **Constants / Literals:** Fixed values that never change (`10`, `3.14`, `'A'`, `"Hello"`).
4. **Operators:** Symbols that perform operations on data (`+`, `-`, `*`, `==`, `&&`, `<<`).
5. **Punctuators / Separators:** Symbols used to structure code (`;`, `{ }`, `( )`, `[ ]`, `,`).

3. Identifiers & Naming Rules

An **Identifier** is the name you give to a variable, function, array, or structure.

- Allowed characters: Letters (`A–Z`, `a–z`), digits (`0–9`), and underscore (`_`).
- **Must begin** with a letter (`A–Z`, `a–z`) or an underscore (`_`). **Cannot start with a digit.**
- **No spaces** or special symbols (`@`, `#`, `$`, `%`, `-`) are allowed.
- **Cannot be a C++ keyword** (like `int`, `float`, `for`).
- **Case-sensitive:** `total`, `Total`, and `TOTAL` are completely different identifiers.

Identifier Example	Valid or Invalid?	Reason
`age`	**Valid**	Starts with a letter, uses only letters
`_value`	**Valid**	Starts with an underscore (`_`)
`total_sum`	**Valid**	Uses letters and underscore
`student1`	**Valid**	Digits are allowed after the first character
`1age`	**Invalid**	Cannot start with a number
`total sum`	**Invalid**	Spaces are not allowed
`@name`	**Invalid**	Special symbol `@` is not allowed
`int`	**Invalid**	Reserved keyword cannot be used as a name

4. Constants & Literals

- **Constants:** Variables whose value is locked and **cannot be changed** after initialization. Declared using the `const` keyword:const double PI = 3.14159;

  const int MAX_MARKS = 100;

- **Literals:** The actual fixed values assigned to variables:
  - **Integer Literals:** `10`, `-45`, `0`
  - **Floating-Point Literals:** `3.14`, `9.8f`, `-0.05`
  - **Character Literals:** Single character inside **single quotes** (`'A'`, `'z'`, `'7'`, `'\n'`)
  - **String Literals:** Text inside **double quotes** (`"Hello World"`, `"Synexis69"`)
  - **Boolean Literals:** `true` (`1`) and `false` (`0`)

5. C++ Keywords (4 - Phase Mastery Roadmap)

Keywords are reserved words built into C++ that have special meaning to the compiler.

Learning Phase	Keywords Included
Phase 1: Beginner Essentials	`int`, `float`, `double`, `char`, `bool`, `void`, `if`, `else`, `switch`, `case`, `default`, `for`, `while`, `do`, `break`, `continue`, `return`, `true`, `false`, `const`, `sizeof`, `using`, `namespace`

Learning Phase	Keywords Included
Phase 2: Core & User-Defined Types	`struct`, `union`, `enum`, `class`, `public`, `private`, `protected`, `new`, `delete`, `this`, `static`, `inline`, `friend`, `typedef`, `auto`, `nullptr`
Phase 3: Advanced OOP & Casting	`virtual`, `override`, `final`, `explicit`, `template`, `typename`, `operator`, `try`, `catch`, `throw`, `noexcept`, `static_cast`, `dynamic_cast`, `const_cast`, `reinterpret_cast`
Phase 4: Systems & Modern C++	`constexpr`, `decltype`, `volatile`, `extern`, `mutable`, `thread_local`, `alignas`, `alignof`, `static_assert`, `concept`, `requires`, `co_await`, `co_yield`, `co_return`

😀 PART - 6

6. 📦Module 6: Data Types in C++

A **Data Type** tells the compiler what kind of value a variable will store and how many bytes of memory to reserve for it.

1. Primitive (Built-in) Data Types

Data Type	Meaning	Memory Size
int	Integer (whole numbers)	4 Bytes (32 bits)
float	Single-precision decimal	4 Bytes (32 bits)
double	Double-precision decimal	8 Bytes (64 bits)
char	Single character (ASCII)	1 Byte (8 bits)
bool	Boolean (Logical state)	1 Byte (8 bits)

Code: Checking Memory Sizes with sizeof()

```
// File: sizeof_demo.cpp - Inspecting memory sizes
#include <iostream>
using namespace std;

int main() {
    cout << "Size of int:    " << sizeof(int) << " bytes\n";
    cout << "Size of float:  " << sizeof(float) << " bytes\n";
    cout << "Size of double: " << sizeof(double) << " bytes\n";
    cout << "Size of char:   " << sizeof(char) << " byte\n";
    cout << "Size of bool:   " << sizeof(bool) << " byte\n";
    return 0;
}
```

```
Size of int:    4 bytes
Size of float:  4 bytes
Size of double: 8 bytes
Size of char:   1 byte
Size of bool:   1 byte
```

〆 🎯MAX VALUE OF AN INTEGER IN (C++) :

```cpp
#include <iostream>
using namespace std;

int main() {
    cout << "MAX OF INT : " << INT_MAX << endl;
    return 0;
}
```

```
MAX OF INT : 2147483647
```

2. Derived Data Types

Derived data types are built from primitive data types:

1. **Array:** Stores multiple values of the **same data type** in continuous memory locations.
    - Example: `int marks[5] = {90, 85, 78, 92, 88};`
2. **Pointer (*):** Stores the **memory address** of another variable.
    - Example: `int a = 10; int *p = &a;` (`&a` gives the address of `a`).
3. **Reference (&):** Creates an **alias (nickname)** for an existing variable.
    - Example: `int a = 10; int &ref = a;` (Changing `ref` directly changes `a`).
4. **Function:** A reusable block of code that performs a specific operation.

3. User-Defined Data Types

Created by the programmer to model custom data:

- **`struct` (Structure):** Groups different data types together in separate memory locations.
- **`union` (Union):** Groups different data types sharing the **same** memory location.
- **`enum` (Enumeration):** Defines a set of named integer constants (like days of the week or menu choices).
- **`class` (Class):** Blueprint for creating objects in Object-Oriented Programming.

4. Empty Data Type (void)

The keyword `void` literally means "**empty**" or "nothing" (no value).

- **Where is `void` used?**
    a. **Void Function Return Type:** When a function performs an action (like printing) but does **not** return a value:void sayHello() {

    cout << "Hello Beginners!\n";

    }

    b. **Empty Parameter List:** `void work(void)` indicates the function takes no inputs.
    c. **Generic Pointer (`void*`):** A special pointer that can hold the memory address of **any data type** (`int`, `float`, `double`).
- **Important Rule:** You **cannot** declare a normal variable of type `void` (writing `void x;` causes a compiler error because the compiler doesn't know how many bytes to allocate).

😯 PART - 7

7. ⚙️ Module 7: Operators in C++

Operators are symbols that tell the compiler to perform mathematical, relational, logical, or bitwise manipulations.

1. Arithmetic Operators (+, -, *, /, %)

Used to perform mathematical calculations:

Operator	Operation Name	Example (a = 10, b = 3)
`+`	Addition	`10 + 3`
`-`	Subtraction	`10 - 3`
`*`	Multiplication	`10 * 3`
`/`	Division (Quotient)	`10 / 3`
`%`	Modulus (Remainder)	`10 % 3`

Code: Arithmetic Operators in Action

```cpp
// File: arithmetic_operators.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 3;
    cout << "Addition (a + b)       = " << (a + b) << endl;
    cout << "Subtraction (a - b)    = " << (a - b) << endl;
    cout << "Multiplication (a * b) = " << (a * b) << endl;
    cout << "Quotient (a / b)       = " << (a / b) << endl;
    cout << "Exact Division         = " << ((float)a / b) << endl;
    cout << "Remainder (a % b)      = " << (a % b) << endl;
    return 0;
}
```

```
Addition (a + b)       = 13
Subtraction (a - b)    = 7
```

```
Multiplication (a * b) = 30
Quotient (a / b)       = 3
Exact Division         = 3.33333
Remainder (a % b)      = 1
```

🧠 2. Six Relational Operators

Used to compare two values. They always return `true` (`1`) or `false` (`0`).

Operator	Meaning	Example
`>`	Greater than	`10 > 5`
`<`	Less than	`10 < 5`
`>=`	Greater than or equal to	`10 >= 10`
`<=`	Less than or equal to	`10 <= 5`
`==`	Equal to	`10 == 10`
`!=`	Not equal to	`10 != 5`

> ⚠️ **Critical Bug Alert: = vs ==**
> Single **=** is the **Assignment Operator** (stores a value). Double **==** is the **Equality Comparison Operator** (checks equivalence). Writing `if (x = 5)` assigns 5 to x and always evaluates to true!

This is the #1 beginner mistake in C++:

- **`= (Single Equals) → GIVE / STORE (Assignment):`** Puts the right-side value into the left-side variable.
  - `int age = 20;` *(Stores 20 inside `age`)*
- **`== (Double Equals) → CHECK / COMPARE (Equality):`** Checks whether both sides are equal.
  - `age == 20` *(Checks if `age` is equal to 20; gives `true` or `false`)*

3. Logical Operators (&&, ||, !)

Used to combine multiple conditions:

Operator	Name	Rule / Meaning
`&&`	**Logical AND**	`true` only if **BOTH** conditions are true
`||`	**Logical OR**	`true` if **AT LEAST ONE** condition is true
`!`	**Logical NOT**	Reverses the result (`true` →

Operator	Name	Rule / Meaning
		`false`)

4. Compound Assignment Operators

Shortcuts for updating a variable:

- `a += 5;` → same as `a = a + 5;`
- `a -= 3;` → same as `a = a - 3;`
- `a *= 2;` → same as `a = a * 2;`
- `a /= 4;` → same as `a = a / 4;`
- `a %= 2;` → same as `a = a % 2;`

Code Practice: Relational, Logical & Assignment Operators

```
// File: operators_practice.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 10, b = 5;
    cout << "(a > b && b > 0): " << (a > b && b > 0) << endl;
    cout << "(a < b || b == 5): " << (a < b || b == 5) << endl;
    cout << "!(a == b): " << !(a == b) << endl;

    a += 5; // a becomes 15
    cout << "After a += 5, a = " << a << endl;
    return 0;
}
```

```
(a > b && b > 0): 1
(a < b || b == 5): 1
!(a == b): 1
After a += 5, a = 15
```

🔢 5. Increment (++) & Decrement (--) Operators

- `++` increases a variable's value by `1`.
- `--` decreases a variable's value by `1`.

Type	Syntax	Rule
Pre-Increment	`++a`	**First increase**, then use the new value
Post-Increment	`a++`	**First use** the old value, then increase
Pre-Decrement	`--a`	**First decrease**, then use the new value
Post-Decrement	`a--`	**First use** the old value, then decrease

Code: Pre vs. Post Increment/Decrement

```cpp
// File: inc_dec_demo.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 5;
    cout << "Initial a: " << a << endl;
    cout << "Post-increment (a++): " << a++ << " (prints 5, then a becomes 6)\n";
    cout << "Now a is: " << a << endl;
    cout << "Pre-increment (++a): " << ++a << " (increases to 7 first, then prints 7)\n";
    return 0;
}
```

```
Initial a: 5
Post-increment (a++): 5 (prints 5, then a becomes 6)
Now a is: 6
Pre-increment (++a): 7 (increases to 7 first, then prints 7)
```

🧠 6. Bitwise Operators (&, |, ^, ~, <<, >>)

Bitwise operators work directly on the **binary bits (0s and 1s)** of integers.
Let a = 5 (Binary 0101) and b = 3 (Binary 0011):

Operator	Name	How It Works
&	Bitwise AND	1 only if both bits are 1
|	Bitwise OR	1 if at least one bit is 1
^	Bitwise XOR	1 if bits are **different**, 0 if same
~	Bitwise NOT	Flips all bits (~x = -(x + 1))
<<	Left Shift	Shifts bits left (a << n = $a \times 2^n$)
>>	Right Shift	Shifts bits right (a >> n = $a \div 2^n$)

Code: Bitwise Operators Program

```cpp
// File: bitwise_demo.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 5, b = 3;
    cout << "a & b  (AND)         = " << (a & b) << endl;
    cout << "a | b  (OR)          = " << (a | b) << endl;
    cout << "a ^ b  (XOR)         = " << (a ^ b) << endl;
    cout << "~a     (NOT)         = " << (~a) << endl;
    cout << "a << 1 (Left Shift)  = " << (a << 1) << endl;
    cout << "a >> 1 (Right Shift) = " << (a >> 1) << endl;
    return 0;
}
```

```
a & b  (AND)         = 1
a | b  (OR)          = 7
a ^ b  (XOR)         = 6
~a     (NOT)         = -6
a << 1 (Left Shift)  = 10
a >> 1 (Right Shift) = 2
```

🔀 7. Ternary (Conditional) Operator (? :)

A clean one-line shorthand for an `if-else` statement.

- **Syntax:**condition ? value_if_true : value_if_false;

Code: Ternary Operator Program

```
// File: ternary_demo.cpp
#include <iostream>
#include <string>
using namespace std;

int main() {
    int a = 15, b = 25;
    int maxVal = (a > b) ? a : b;
    cout << "Maximum value is: " << maxVal << endl;

    string result = (a % 2 == 0) ? "Even" : "Odd";
    cout << a << " is " << result << endl;
    return 0;
}
```

```
Maximum value is: 25
15 is Odd
```

🔀

😬 PART - 8

8. Exercise: Triangle Area & Perimeter (Heron's Formula)

Formula (Heron's Formula):

- Perimeter = a + b + c
- Semi-perimeter (s) = (a + b + c) / 2.0
- `Area = √(s × (s − a) × (s − b) × (s − c))` *(requires `#include <cmath>`)*

```cpp
// File: herons_formula.cpp
#include <iostream>
#include <cmath>
using namespace std;

int main() {
    double a, b, c;
    cout << "Enter three sides of the triangle: ";
    cin >> a >> b >> c;

    double perimeter = a + b + c;
    double s = perimeter / 2.0;
    double area = sqrt(s * (s - a) * (s - b) * (s - c));

    cout << "Perimeter of Triangle = " << perimeter << endl;
    cout << "Area of Triangle      = " << area << endl;
    return 0;
}
```

```
Enter the three sides of the triangle: 3 4 5
Perimeter of Triangle = 12
Area of Triangle      = 6
```

---

8. 🔀 Module 8: Process Flow & Selection Statements (if-else & Menu Programs)

1. Three Types of Process Flow in C++

1. **Sequential Flow:** Statements execute one after another from top to bottom.
2. **Selection (Conditional) Flow:** Executes a block of code only if a condition is true (`if`, `if-else`, `else-if`, `switch`).
3. **Iterative (Looping) Flow:** Repeats a block of code multiple times as long as a condition holds (`while`, `for`, `do-while`).

2. Selection Statement Syntaxes

(a) Simple `if` Statement

```
// --- (a) Simple if Statement ---
if (condition) {
    // Executes only if condition is true
}
```

(b) `if - else` Statement

```
// --- (b) if-else Statement ---
if (condition) {
    // Executes if condition is true
} else {
    // Executes if condition is false
}
```

(c) `if - else if - else` Ladder

```
// --- (c) if-else if-else Ladder ---
if (condition1) {
    // Executes if condition1 is true
} else if (condition2) {
```

```
    // Executes if condition2 is true
} else {
    // Executes if all above conditions are false
}
```

🔧 3. Essential Selection C++ Programs

1Q) Find Bigger Value Between Two Numbers

```
📄// File: bigger_of_two.cpp
#include <iostream>
using namespace std;

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;

    if (a > b) {
        cout << a << " is bigger than " << b << endl;
    } else if (b > a) {
        cout << b << " is bigger than " << a << endl;
    } else {
        cout << "Both numbers are equal!" << endl;
    }
    return 0;
}
```

```
⚙️
📄Enter two numbers: 45 20
45 is bigger than 20
```

🔧 2Q) Check Multiple (e.g., Multiple of 7)

- **Logic:** A number `n` is a multiple of `7` if dividing `n` by `7` leaves a remainder of `0` (`n % 7 == 0`).

```
📄// File: multiple_of_7.cpp
#include <iostream>
```

```
using namespace std;

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;

    if (n % 7 == 0) {
        cout << n << " is a multiple of 7." << endl;
    } else {
        cout << n << " is NOT a multiple of 7." << endl;
    }
    return 0;
}
```

```
Enter a number: 49
49 is a multiple of 7.
```

3Q) Triangle Classifier (Equilateral, Isosceles, or Scalene)

- **Logic:**
  - First check if the 3 sides can form a valid triangle: `(a + b > c) && (b + c > a) && (c + a > b)`.
  - **Equilateral Triangle:** All three sides are equal (`a == b && b == c`).
  - **Isosceles Triangle:** Any two sides are equal (`a == b || b == c || c == a`).
  - **Scalene Triangle:** All three sides are different.

```
// File: triangle_classifier.cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cout << "Enter three sides of the triangle: ";
    cin >> a >> b >> c;
```

```
    if ((a + b > c) && (b + c > a) && (c + a > b)) {
        if (a == b && b == c) {
            cout << "Equilateral Triangle (All 3 sides are equal)
\n";
        } else if (a == b || b == c || c == a) {
            cout << "Isosceles Triangle (2 sides are equal)\n";
        } else {
            cout << "Scalene Triangle (All 3 sides are different)
\n";
        }
    } else {
        cout << "Invalid Triangle! The sides cannot form a triangle.
\n";
    }
    return 0;
}
```

```
Enter three sides of the triangle: 5 5 8
Isosceles Triangle (2 sides are equal)
```

4Q) Menu-Driven Calculator (using if - else if)

```
// File: menu_calculator.cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, ch;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    cout << "\n** MENU **\n";
    cout << "1. Addition\n2. Subtraction\n3. Multiplication\n4.
Quotient\n5. Remainder\n";
    cout << "ENTER YOUR CHOICE: ";
    cin >> ch;

    if (ch == 1) {
        cout << "Addition = " << (a + b) << endl;
    } else if (ch == 2) {
```

```
        cout << "Subtraction = " << (a - b) << endl;
    } else if (ch == 3) {
        cout << "Multiplication = " << (a * b) << endl;
    } else if (ch == 4) {
        cout << "Quotient = " << (a / b) << endl;
    } else if (ch == 5) {
        cout << "Remainder = " << (a % b) << endl;
    } else {
        cout << "Invalid choice..." << endl;
    }
    return 0;
}
```

```
Enter two numbers: 20 6
** MENU **
1. Addition
2. Subtraction
3. Multiplication
4. Quotient
5. Remainder
ENTER YOUR CHOICE: 5
Remainder when 20 is divided by 6 is 2
```

5Q) Greatest of Three Numbers

```
// File: greatest_of_three.cpp
#include <iostream>
using namespace std;

int main() {
    int a, b, c;
    cout << "Enter three numbers: ";
    cin >> a >> b >> c;

    if (a >= b && a >= c) {
        cout << "Greatest number is: " << a << endl;
    } else if (b >= a && b >= c) {
        cout << "Greatest number is: " << b << endl;
    } else {
```

```
        cout << "Greatest number is: " << c << endl;
    }
    return 0;
}
```

```
Enter three numbers: 25 89 42
Greatest number is: 89
```

🫠 PART - 9

9. 🔄 Module 9: Iterative Process Flow — while Loops

1. What is a Loop & 3 Essential Parts

A **Loop** is used to execute a block of statements repeatedly as long as a given condition is `true`.

1. **Initialization:** Setting the starting value of the loop counter (e.g., `int i = 1;`).
2. **Condition:** The test checked before each repetition (e.g., `while (i <= 10)`).
3. **Update (Increment/Decrement):** Changing the loop variable so the loop eventually stops (e.g., `i++;`).

1.

```
// ''--- Standard while Loop Syntax ---
initialization;             // 1. Starting counter
while (condition) {         // 2. Loop continuation condition
    // Statements to repeat
    update;                 // 3. Counter step (i++, i--, etc.)
}
```

🎋 2. Practice Programs Using while Loops

Program 1: Print Numbers 1 to 10

2.

```
// File: print_1_to_10.cpp
#include <iostream>
using namespace std;

int main() {
    int i = 1;
    while (i <= 10) {
        cout << i << " ";
        i++;
    }
    cout << endl;
    return 0;
}
```

3. ㅎ

ㄲ1 2 3 4 5 6 7 8 9 10

ㅆ Program 2: Print Odd Numbers from 1 to N

4.

ㄲ
```cpp
// File: odd_numbers_1_to_n.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter N: ";
    cin >> n;
    int i = 1;
    while (i <= n) {
        cout << i << " ";
        i += 2; // Jump by 2
    }
    cout << endl;
    return 0;
}
```

5. ㅎ

ㄲEnter N: 15
1 3 5 7 9 11 13 15

ㅆ Program 3: Multiplication/Division Step Loops & Factorial

6.

ㄲ
```cpp
// File: mult_step_loop.cpp - i = i * 2 update
#include <iostream>
using namespace std;

int main() {
    int i = 1;
    while (i <= 64) {
        cout << i << " ";
        i = i * 2;
    }
    cout << endl;
    return 0;
```

```
}
```

7. ㅎ

```
1 2 4 8 16 32 64
```

8. ㅎ

```
// File: div_step_loop.cpp - i = i / 2 update
#include <iostream>
using namespace std;

int main() {
    int i = 100;
    while (i >= 1) {
        cout << i << " ";
        i = i / 2;
    }
    cout << endl;
    return 0;
}
```

9. ㅎ

```
100 50 25 12 6 3 1
```

10. ㅎ

```
// File: factorial_while.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    long long fact = 1;
    int i = 1;
    while (i <= n) {
        fact *= i;
        i++;
    }
    cout << "Factorial of " << n << " is: " << fact << endl;
    return 0;
}
```

11. ㅎ

```
Enter a number: 5
```

```
Factorial of 5 is: 120
```

ㅆ Program 4: Sum of Numbers from 1 to N

12.

```
ㄲ// File: sum_1_to_n.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter N: ";
    cin >> n;
    int sum = 0, i = 1;
    while (i <= n) {
        sum += i;
        i++;
    }
    cout << "Sum of numbers from 1 to " << n << " = " << sum <<
endl;
    return 0;
}
```

13. ㅎ

```
ㄲEnter N: 10
Sum of numbers from 1 to 10 = 55
```

ㅆ Program 5: Sum of Odd Numbers from 1 to N

14.

```
ㄲ// File: sum_odd_1_to_n.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter N: ";
    cin >> n;
    int sum = 0, i = 1;
    while (i <= n) {
        sum += i;
        i += 2;
```

```
    }
    cout << "Sum of odd numbers up to " << n << " = " << sum <<
endl;
    return 0;
}
```

15. ㅎ

```
Enter N: 10
Sum of odd numbers up to 10 = 25
```

Program 6: Find Odd/Even Numbers and Separate Sums

16.

```
// File: odd_even_sums.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter N: ";
    cin >> n;
    int i = 1, evenSum = 0, oddSum = 0;
    while (i <= n) {
        if (i % 2 == 0) {
            cout << i << " is Even\n";
            evenSum += i;
        } else {
            cout << i << " is Odd\n";
            oddSum += i;
        }
        i++;
    }
    cout << "Sum of Even numbers = " << evenSum << endl;
    cout << "Sum of Odd numbers  = " << oddSum << endl;
    return 0;
}
```

17. ㅎ

```
Enter N: 5
1 is Odd
2 is Even
3 is Odd
```

```
4 is Even
5 is Odd
Sum of Even numbers = 6
Sum of Odd numbers  = 9
```

🔹 Program 7: Reverse Order Loop (N down to 1) & Infinite Loop Warning

18.

```cpp
// File: reverse_countdown.cpp
#include <iostream>
using namespace std;

int main() {
    int n = 10;
    cout << "Reverse countdown from " << n << " to 1:\n";
    int i = n;
    while (i >= 1) {
        cout << i << " ";
        i--; // Decrement counter
    }
    cout << endl;
    return 0;
}
```

19. ㅎ

```
Reverse countdown from 10 to 1:
10 9 8 7 6 5 4 3 2 1
```

🔹 Program 8: Print Multiplication Table

20.

```cpp
// File: multiplication_table.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number for multiplication table: ";
    cin >> n;
    int i = 1;
    while (i <= 10) {
        cout << n << " x " << i << " = " << (n * i) << endl;
```

```
        i++;
    }
    return 0;
}
```

21. ㅎ

```
ㄲEnter a number for multiplication table: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
...
7 x 10 = 70
```

ㅎ

😰 PART - 10

10. 🎯 Module 10: for Loops & Number Logic

1. for Loop Syntax & Execution Flow

1.

```
// --- Compact for Loop Syntax ---
for (initialization; condition; update) {
    // Loop body statements
}
```

2. **Initialization** runs **only once** at the very start (`int i = 1`).
3. **Condition** is checked (`i <= n`). If `true`, enter the loop body; if `false`, exit the loop.
4. **Loop Body** executes.
5. **Update** runs (`i++`), and then flow goes back to **Step 2 (Condition)**.

2. Practice Programs Using for Loops

Program 1: Find All Factors (Divisors) of a Number N

- **Logic:** A number `i` is a factor of `n` if `n % i == 0`.

```
// File: factors_of_n.cpp
#include <iostream>
using namespace std;

int main() {
    int n;
    cout << "Enter a number: ";
    cin >> n;
    cout << "Factors of " << n << " are: ";
    for (int i = 1; i <= n; i++) {
        if (n % i == 0) {
            cout << i << " ";
        }
    }
    cout << endl;
```

```
    return 0;
}
```

```
Enter a number: 12
Factors of 12 are: 1 2 3 4 6 12
```

Program 2: Check Perfect Number

- **What is a Perfect Number?** A number whose sum of proper divisors (factors excluding the number itself) is equal to the original number.
    - *Example 1:* `6 → factors excluding 6 are 1, 2, 3 → 1 + 2 + 3 = 6` (**Perfect Number**).
    - *Example 2:* `28 → factors excluding 28 are 1, 2, 4, 7, 14 → 1 + 2 + 4 + 7 + 14 = 28` (**Perfect Number**).

```
// File: perfect_number.cpp
#include <iostream>
using namespace std;

int main() {
    int n, sum = 0;
    cout << "Enter a positive number: ";
    cin >> n;
    for (int i = 1; i < n; i++) {
        if (n % i == 0) {
            sum += i;
        }
    }
    if (sum == n) {
        cout << n << " is a Perfect Number!" << endl;
    } else {
        cout << n << " is NOT a Perfect Number." << endl;
    }
    return 0;
}
```

```
Enter a positive number: 6
6 is a Perfect Number!
```

ত

🥱 PART - 11

11. 🎨 Module 11: Nested Loops & Pattern Printing

1. What is a Nested Loop?

A **Nested Loop** is a loop inside another loop.

- **Outer Loop (`i`):** Controls the **Rows** (horizontal lines).
- **Inner Loop (`j`):** Controls the **Columns** (what gets printed inside each row).
- For every 1 iteration of the outer loop, the inner loop runs completely from start to finish!

2. Important Star & Number Pattern Programs

Pattern 1: Solid Square Star Pattern (N x N)

```
// File: pattern_square.cpp
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    for (int i = 1; i <= n; i++) {        // Outer loop -> Rows
        for (int j = 1; j <= n; j++) {    // Inner loop -> Columns
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

```
* * * *
* * * *
* * * *
* * * *
```

🔺 Pattern 2: Right-Angled Triangle Star Pattern

```cpp
// File: pattern_right_triangle.cpp
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

```
*
* *
* * *
* * * *
```

🔺 Pattern 3: Right-Angled Number Triangle

```cpp
// File: pattern_number_triangle.cpp
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= i; j++) {
            cout << j << " ";
        }
        cout << endl;
    }
    return 0;
}
```

```
1
1 2
1 2 3
1 2 3 4
```

Pattern 4: Inverted Right-Angled Triangle Pattern

```cpp
// File: pattern_inverted_triangle.cpp
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    for (int i = n; i >= 1; i--) {
        for (int j = 1; j <= i; j++) {
            cout << "* ";
        }
        cout << endl;
    }
    return 0;
}
```

```
* * * *
* * *
* *
*
```

Pattern 5: Centered Star Pyramid Pattern

```cpp
// File: pattern_pyramid.cpp
#include <iostream>
using namespace std;

int main() {
    int n = 4;
    for (int i = 1; i <= n; i++) {
        for (int s = 1; s <= n - i; s++) cout << "  ";
        for (int j = 1; j <= (2 * i - 1); j++) cout << "* ";
        cout << endl;
```

```
    }
    return 0;
}
```

```
    *
  * * *
* * * * *
* * * * * * *
```

Pattern 6: Hollow U-Shape Boundary Star Pattern

```
// File: pattern_hollow_u.cpp
#include <iostream>
using namespace std;

int main() {
    int n = 5;
    for (int i = 1; i <= n; i++) {
        for (int j = 1; j <= n; j++) {
            if (j == 1 || j == n || i == n) {
                cout << "* ";
            } else {
                cout << "  ";
            }
        }
        cout << endl;
    }
    return 0;
}
```

```
*       *
*       *
*       *
* * * * *
```

🧮 Program 7: Compute Binomial Coefficient (nCr) Using Loops

- **Formula:** nCr = n! / (r! × (n - r)!)

```
📄// File: ncr_loops.cpp
#include <iostream>
using namespace std;

int main() {
    int n, r;
    cout << "Enter n and r: ";
    cin >> n >> r;
    long long factN = 1, factR = 1, factNR = 1;
    for (int i = 1; i <= n; i++) factN *= i;
    for (int i = 1; i <= r; i++) factR *= i;
    for (int i = 1; i <= (n - r); i++) factNR *= i;
    long long nCr = factN / (factR * factNR);
    cout << "Binomial Coefficient (" << n << "C" << r << ") = " <<
nCr << endl;
    return 0;
}
```

```
🖥️
📄Enter n and r: 5 2
Binomial Coefficient (5C2) = 10
🖥️
```

😵 PART - 12

12. 🛠️ Module 12: Functions in C++ & Greatest Common Divisor (GCD)

1. What is a Function & Its 3 Parts

A **Function** is a self-contained, reusable block of code that performs a specific task. Instead of repeating the same loop 3 times (like we did for `nCr` above), we can write **one** `factorial()` function and call it whenever needed!

1. **Function Declaration (Prototype):** Tells the compiler the function's name, return type, and parameter types.
2. **Function Definition:** The actual body of the function containing the statements.
3. **Function Call:** Calling the function by name inside `main()`.

Code: Computing nCr Cleanly Using a Function

```cpp
// File: ncr_function.cpp
#include <iostream>
using namespace std;

// Function Definition: calculates factorial of integer x
long long factorial(int x) {
    long long fact = 1;
    for (int i = 1; i <= x; i++) {
        fact *= i;
    }
    return fact;
}

int main() {
    int n, r;
    cout << "Enter n and r: ";
    cin >> n >> r;
    long long nCr = factorial(n) / (factorial(r) * factorial(n - r));
    cout << n << "C" << r << " = " << nCr << endl;
    return 0;
```

```
}
```

🧮 2. Greatest Common Divisor (GCD / HCF)

The **GCD (Greatest Common Divisor)** of two numbers a and b is the largest positive integer that divides both a and b without leaving a remainder.

Method 1: Euclidean Algorithm (Modulo Method — Fastest)

```
📄// File: gcd_euclidean.cpp
#include <iostream>
using namespace std;

int findGCD(int a, int b) {
    while (b != 0) {
        int remainder = a % b;
        a = b;
        b = remainder;
    }
    return a;
}

int main() {
    int a, b;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    cout << "GCD of " << a << " and " << b << " is: " << findGCD(a,
b) << endl;
    return 0;
}
```

```
⚙
📄Enter two numbers: 56 98
GCD of 56 and 98 is: 14
```

🧮 Method 2: Simple Loop Method

```
📄// File: gcd_loop.cpp
#include <iostream>
```

```cpp
using namespace std;

int main() {
    int a, b, gcd = 1;
    cout << "Enter two numbers: ";
    cin >> a >> b;
    int limit = (a < b) ? a : b;
    for (int i = 1; i <= limit; i++) {
        if (a % i == 0 && b % i == 0) {
            gcd = i;
        }
    }
    cout << "GCD = " << gcd << endl;
    return 0;
}
```

ㅎ

🤐 PART - 13

13. 🎛️ Module 13: switch Statement & setprecision Formatting

1. C++ switch Statement & Key Rules

The `switch` statement is a clean multi-way branch statement used when comparing a single variable against multiple fixed integer or character values (`case`).

- Works only with **integral types** (`int`, `char`, `enum`) — **not** `float`, `double`, or `string`.
- **`break` statement:** Stops execution and exits the `switch` block. If you forget `break`, execution "falls through" into the next `case`!
- **`default` case:** Runs when none of the `case` values match (similar to `else`).

Code: Menu Calculator Using switch-case

```cpp
// File: switch_calculator.cpp
#include <iostream>
using namespace std;

int main() {
    int a = 12, b = 5, choice;
    cout << "Numbers: a = " << a << ", b = " << b << endl;
    cout << "1.Add  2.Subtract  3.Multiply  4.Quotient
5.Remainder\n";
    cout << "Enter choice (1-5): ";
    cin >> choice;

    switch (choice) {
        case 1:
            cout << "Addition = " << (a + b) << endl;
            break;
        case 2:
            cout << "Subtraction = " << (a - b) << endl;
            break;
        case 3:
            cout << "Multiplication = " << (a * b) << endl;
            break;
```

```
        case 4:
            cout << "Quotient = " << (a / b) << endl;
            break;
        case 5:
            cout << "Remainder = " << (a % b) << endl;
            break;
        default:
            cout << "Invalid choice!" << endl;
    }
    return 0;
}
```

🧮 2. Setting Decimal Precision (setprecision in <iomanip>)

To control how many digits are printed after the decimal point for `float` or `double`:

- Include the `<iomanip>` header: `#include <iomanip>`
- Use `fixed << setprecision(n)` to print exactly `n` digits after the decimal point.

```
📄// File: precision_demo.cpp
#include <iostream>
#include <iomanip>
using namespace std;

int main() {
    double pi = 3.14285714285;
    cout << "Default output: " << pi << endl;
    cout << fixed << setprecision(2) << "2 decimal places: " << pi
<< endl;
    cout << fixed << setprecision(4) << "4 decimal places: " << pi
<< endl;
    cout << fixed << setprecision(0) << "0 decimal places: " << pi
<< endl;
    return 0;
}
```

```
Default output: 3.14286
2 decimal places: 3.14
4 decimal places: 3.1429
0 decimal places: 3
```

🥺 PART - 14

14. 🏛️ Module 14: User-Defined Types — struct, union, and enum

1. Structure (struct) in C++

A **Structure (`struct`)** is a user-defined data type that groups variables of **different data types** under a single name.

- Each member inside a `struct` gets its **own separate memory space**.
- Members are accessed using the **dot operator (`.`)**.

Program 1: Basic Student Structure

```
// File: struct_student.cpp
#include <iostream>
#include <string>
using namespace std;

struct Student {
    int id;
    string name;
    float marks;
};

int main() {
    Student s1;
    s1.id = 101;
    s1.name = "Sahil";
    s1.marks = 95.5f;

    cout << "Student ID    : " << s1.id << endl;
    cout << "Student Name  : " << s1.name << endl;
    cout << "Student Marks : " << s1.marks << endl;
    return 0;
}
```

```
Student ID    : 101
Student Name  : Sahil
Student Marks : 95.5
```

Program 2: Structure with User Input (getline)

```cpp
// File: struct_employee.cpp
#include <iostream>
#include <string>
using namespace std;

struct Employee {
    int empId;
    string fullName;
    double salary;
};

int main() {
    Employee e;
    cout << "Enter Employee ID: ";
    cin >> e.empId;
    cin.ignore(); // Clears newline from buffer
    cout << "Enter Full Name: ";
    getline(cin, e.fullName);
    cout << "Enter Salary: ";
    cin >> e.salary;

    cout << "\n--- Employee Details ---\n";
    cout << "ID     : " << e.empId << "\n";
    cout << "Name   : " << e.fullName << "\n";
    cout << "Salary : " << e.salary << "\n";
    return 0;
}
```

2. Union (union) in C++

A **Union (`union`)** is a user-defined data type similar to a `struct`, except **all members share the exact same memory location**.

- **Memory Size:** The size of a `union` is equal to the size of its **largest member**.
-

> 💡 **Pro Tip: Struct vs Union Memory Layout**
> A **`struct`** allocates separate memory for every member (total size = sum of sizes). A **`union`** shares the same memory location for all members (total size = size of largest member). Only one union member holds a valid value at a time!

- **Key Behavior:** Because all members share the same memory, **only one member can store a value at any given time**. Assigning a value to a new member overwrites the previous value.

Structure vs. Union Comparison

Feature	Structure (struct)	Union (union)
Keyword	`struct`	`union`
Memory Allocation	Each member gets its **own separate** memory	All members **share the same** memory location
Total Size	Sum of sizes of all members (plus padding)	Size of the **largest** member only
Active Members	All members can hold values simultaneously	Only **one** member can hold a valid value at a time

Code: Union Demonstration in C++

```cpp
// File: union_demo.cpp
#include <iostream>
using namespace std;

union Data {
    int intVal;
    float floatVal;
    char charVal;
};

int main() {
    Data d;
    cout << "Size of union Data: " << sizeof(Data) << " bytes\n";

    d.intVal = 65;
    cout << "intVal   = " << d.intVal << endl;
```

```
    d.floatVal = 99.25f; // Overwrites shared memory
    cout << "floatVal = " << d.floatVal << endl;

    d.charVal = 'A';     // Overwrites shared memory again
    cout << "charVal  = " << d.charVal << endl;
    return 0;
}
```

```
Size of union Data: 4 bytes
intVal   = 65
floatVal = 99.25
charVal  = A
```

3. Enumeration (enum & enum class)

An **Enumeration (`enum`)** is a user-defined data type that consists of a fixed set of **named integer constants**. It makes code much easier to read and maintain.

- By default, `enum` values start at `0` and increase by `1` (`0, 1, 2...`), or you can assign custom integer values.

Program 1: Basic enum with Assigned Values

```
// File: enum_basic.cpp
#include <iostream>
using namespace std;

enum Day {
    MONDAY = 1, TUESDAY = 2, WEDNESDAY = 3, THURSDAY = 4, FRIDAY = 5
};

int main() {
    Day today = WEDNESDAY;
    cout << "Day value: " << today << endl;
    if (today == MONDAY) cout << "Start of the week\n";
    else if (today == WEDNESDAY) cout << "Midweek\n";
    else if (today == FRIDAY) cout << "Almost weekend\n";
    else cout << "Another weekday\n";
```

```
    return 0;
}
```

```
Day value: 3
Midweek
```

Program 2: enum with switch-case (Menu Example)

```
// File: enum_switch.cpp
#include <iostream>
using namespace std;

enum Menu { HOME = 1, PROFILE, SETTINGS, LOGOUT };

int main() {
    int choice;
    cout << "===== MENU =====\n1. Home\n2. Profile\n3. Settings\n4.
Logout\nEnter choice: ";
    cin >> choice;

    Menu option = static_cast<Menu>(choice);
    switch (option) {
        case HOME:     cout << "You are on Home page.\n"; break;
        case PROFILE:  cout << "You are on Profile page.\n"; break;
        case SETTINGS: cout << "You are on Settings page.\n"; break;
        case LOGOUT:   cout << "You are logging out.\n"; break;
        default:       cout << "Invalid choice!\n";
    }
    return 0;
}
```

```
===== MENU =====
1. Home
2. Profile
3. Settings
4. Logout
Enter choice: 2
You are on the Profile page.
```

🚦 Program 3: Scoped Enum (enum class) — Traffic Light Example

- **Why `enum class`?** Modern C++ provides `enum class` for stronger type safety—it prevents accidental conversion to `int` and avoids name collisions by using the scope resolution operator (`::`).

```
// File: enum_class_traffic.cpp
#include <iostream>
#include <string>
using namespace std;

enum class TrafficLight { Red, Yellow, Green };

string getLightName(TrafficLight light) {
    switch (light) {
        case TrafficLight::Red:    return "Red";
        case TrafficLight::Yellow: return "Yellow";
        case TrafficLight::Green:  return "Green";
        default:                   return "Unknown";
    }
}

int main() {
    TrafficLight light = TrafficLight::Green;
    cout << "Current signal: " << getLightName(light) << endl;
    if (light == TrafficLight::Green) {
        cout << "You can Go!\n";
    }
    return 0;
}
```

```
Current signal: Green
You can Go!
```

📝 Quick Exam Checklist & High-Yield Concepts

- **10 Golden Rules Check:** Semicolons required, case-sensitivity active, entry point `int main()`,

always initialize local counters, integer division drops decimal, `=` vs `==`, modulus `%` only for integers, match braces `{ }`, 0-based array indexing, clear comments.

- **Data Type Sizes:** `char` (1 B), `bool` (1 B), `int` (4 B), `float` (4 B), `double` (8 B). Inspect with `sizeof()`.
- **Loop Pitfall:** Always verify loop counter updates (`i++` or `i--`) to prevent infinite execution loops.
- **Pre vs Post Inc/Dec:** `++a` updates then yields value; `a + +` yields current value then updates.
- **Struct vs Union:** Structs allocate distinct memory for all members; Unions share one memory block sized to the largest member.
- **Scoped Enums:** Prefer `enum class` over plain `enum` in modern C++ for strong type safety and scope resolution.